Arbitrary Arithmetic

H Vasant Kumar
CS23BTECH11029
Software Development Fundamentals

Objective

- Implementation of an infinite/arbitrary-precision arithmetic library in Java using OOP concepts.
- If you are given two strings as input, you need to add support for addition, subtraction, multiplication, and division, for both integer and float data types.
- Ensure efficient memory management when dealing with large integers and floating-point numbers, especially when performing operations on large datasets or long-precision numbers.
- Implement a mechanism to define the precision of the arithmetic operations.
- Safely handle very large numbers and overflow scenarios.

Library Specification

It has a package called `arbitraryarithmetic` which consists of two classes `AInteger` and `AFloat`.

AInteger.java / Integer class

I created a directory named `arbitraryarithmetic`. Inside this directory, I created the file `AInteger.java`. To make it a package, I included the statement `package arbitraryarithmetic;` at the top of the file.

Properties or members

It has a protected member named `'num'` of type String.

Constructors

- Default constructor `AInteger()` that initializes the instance with value 0.
- Constructor `AInteger(String num)` that initializes the instance by the number whose string representation is given by 'num'.
- Copy constructor that creates an instance of `AInteger`.

Methods / Functions

`parse(String num)` A static function that returns an instance of `AInteger` class.

`get()` Returns the object's member which is "num".

`set(String num)` Sets the object's member which is "num".

`remove_leading_zero(String num)` Removes the extra unnecessary leading zeroes.

`add(AInteger other)` Adds integers of arbitrary length.

`sub(AInteger other)` Subtracts integers of arbitrary length.

`mul(AInteger other)` Multiplies integers of arbitrary length.

`div(AInteger other)` Divides integers of arbitrary length.

Deeper into the methods

`add(AInteger other)`

- I first eliminated leading zeros from both operands to ensure accurate numerical representation and comparison.
- This function adds any two integers, whether positive or negative.
- I initially ensured it worked correctly for positive integers.
- I Implemented manual addition by iterating through digits from right to left (least to most significant), simulating arithmetic by hand.
- The numbers are processed in a while loop, where each digit is added starting from the unit's place.
- Manages and propagates carry values accurately during each step of the digit-wise addition process.
- Accommodates operands of different lengths by continuing the addition with remaining digits of the longer operand once the shorter is exhausted.
- Accumulates the result in reverse order and subsequently reverses it to obtain the final correctly ordered result.
- Handles any leftover carry after completing digit-wise addition by appending it to the final result string.
- Prepends a negative sign to the result if both original operands were negative, preserving mathematical correctness.
- It initially detects scenarios where the operands have opposing signs and transforms the operation into a subtraction of absolute values.
- Finally, I passed the result to a function that removes any leading zeros.
- Returns a new `AInteger` object encapsulating the final computed result, conforming to the class's design contract.
- Supports addition of arbitrarily large integers by performing all operations at the string level, circumventing native data type limitations.

`sub(AInteger other)`

- First, removes any leading zeroes from both numbers to make sure the values are clean and accurate.
- As with addition, I initially implemented subtraction for positive integers.
- If both numbers are exactly the same, the result is zero.
- I compared the inputs to determine whether the result should be positive or negative.
- Stores a flag to keep track of whether the result should have a minus sign or not
- The subtraction was performed by subtracting the smaller number from the larger one.
- Performs subtraction digit by digit starting from the rightmost digit (like in manual subtraction)
- If a digit in the first number is smaller than the digit being subtracted, it borrows from the next digit.
- Keeps subtracting digits until all digits in both numbers are used up.

- Reverses the result string since digits were added from right to left during subtraction.
- Depending on the earlier comparison, I prefixed the result with a minus sign if necessary.
- If both inputs were negative, I simply reversed the order of the operands before performing the subtraction.
- If the first number is negative and the second is positive, it adds the magnitudes and puts a minus sign in front of the result.
- If the first number is positive and the second is negative, it simply adds the two magnitudes (because subtracting a negative is like adding).
- Finally, I passed the result to a function that removes any leading zeros.
- Creates and returns a new `AInteger` object with the final result.

`mul(AInteger other)`

- Removes any leading zeroes from both input numbers to clean them up.
- If either number is zero, the result is immediately zero.
- I initially implemented multiplication for positive integers, building upon the logic used for addition and subtraction.
- Initializes an empty result that will hold the final answer.
- Loops through each digit of the second number from left to right.
- For each digit
  - Converts the character digit to an integer
  - Creates a temporary result by adding the first number to itself 'digit' number of times.
  - Appends the appropriate number of zeroes at the end to simulate multiplication by powers of 10.
- Adds the result of each digit's multiplication to the running total.
- If both input numbers were negative, I multiplied their absolute values.
- If only one of the inputs was negative, I multiplied their absolute values and prefixed the result with a minus sign.
- Finally, the result was passed through a function to remove any leading zeros before returning it.
- Returns the result wrapped inside a new `AInteger` object.

`div(AInteger other)`

- As with other arithmetic operations, I first removed any leading zeros from the inputs.
- Then, I removed the negative signs so it can just work with magnitudes.
- I checked if the divisor was zero and, if so, threw an error to handle division by zero.
- If the dividend was zero, I immediately returned zero as the result.
- If divisor is 1, the result is just the dividend.
- If both are equal, the result is 1.
- If divisor is larger than dividend, result is 0.

- I then implemented the long division algorithm for performing the division.
- I take one digit at a time from the dividend:
- The code builds a string `comp` to represent a part of the dividend being divided.
- For each digit in the dividend:
  - Adds the digit to `comp`.
  - If `comp` is still smaller than the divisor, adds "0" to the result and continues.
  - Otherwise, finds the largest digit (0 to 9) such that divisor $\times$ digit $\leq$ `comp`
  - Adds that digit to the final answer string `fin`.
  - Subtracts divisor $\times$ digit from `comp` and updates it.
- If both input numbers were negative, I divided their absolute values.
- If only one of the inputs was negative, I divided their absolute values and prefixed the result with a minus sign.
- Finally, the result was passed through a function to remove any leading zeros before returning it.
- I wrapped the result into a new `AInteger` and returns it.

AFloat.java / Float class

I created the file `AFloat.java` inside the `arbitraryarithmetic` directory. To include it in the same package, I added the statement `package arbitraryarithmetic;` at the top of the file.

Properties or Members

It has a protected member named `'num'` of type String.

Constructors

- Default constructor `AFloat()` that initializes the instance with value 0.0.
- Constructor `AFloat(String num)` that initializes the instance by the number whose string representation is given by 'num'.
- Copy constructor that creates an instance of `AFloat`.

Methods / Functions

`parse(String num)` A static function that returns an instance of `AFloat` class.

`get()` Returns the member of the object.

`set(String num)` Sets the member of the object to `num`.

`decimal_digits()` Returns the number of decimal digits of their member.

`decimal_point()` Confirms whether the member has decimal point.

`remove_extra_zeroes(String str)` Function which removes extra zeroes at the end of decimal places.

`add(AFloat other)` Adds any two floating-point numbers of arbitrary length.

`sub(AFloat other)` Subtracts any two floating-point numbers of arbitrary length.

`mul(AFloat other)` Multiplies any two floating-point numbers of arbitrary length.

`div(AFloat other)` Divides any two floating-point numbers of arbitrary length.

`truncate(String str)` Truncates the result to 30 decimal places.

A deeper look into methods

`add(AFloat other)`

- It implements the `add()` method in `AFloat.java` to perform addition of two floating-point numbers represented as strings.
- First, It ensures both numbers are in a proper floating-point format
- If a number didn't already contain a decimal point, It appends one to ensure consistent processing.
- It calculates the decimal lengths of both operands using the `decimal_digits()` method.
- If both operands had the same number of digits after the decimal:
  - It removes the decimal points and adds the integer parts of the numbers.
  - The result is then formatted by inserting the decimal point at the correct position based on the original decimal length.
- If operand 1 has a greater decimal length than operand 2:
  - It pads operand 2 with zeros to match the decimal length of operand 1.
  - The result is calculated in the same way as the equal decimal length case.
- If operand 2 has a greater decimal length than operand 1:
  - It pads operand 1 with zeros to match the decimal length of operand 2.
  - The result is calculated in the same way as the equal decimal length case.
- After the addition of the integer parts, it handles the following:
  - If the result has more digits than the original decimal length, it inserts the decimal point at the proper position.
  - If the result has fewer digits, it adds the necessary number of leading zeros before the decimal point.
  - If the result is negative, the negative sign is handled and inserted properly.
- After constructing the result string, It removes any unnecessary leading zeroes and uses the `remove_extra_zeroes()` and `truncate()` functions to clean up the result before returning it.
- Finally, it wraps the result into a new `AFloat` object and returns it.

`sub(AFloat other)`

- Convert both operands to strings and ensure they have a decimal point.
- If either operand doesn't have a decimal point, it adds one.
- It calculates the decimal lengths of both operands using the `decimal_digits()` method.
- If both operands have equal decimal lengths:
  - It removes the decimal points and performs subtraction by treating the numbers as integers.

  - The result is then formatted, ensuring the decimal point is inserted correctly based on the original decimal length.
  - If the result has more digits than the decimal length, the decimal point is inserted at the correct position.
  - If the result is negative, the negative sign is handled properly and inserted in the correct place.
  - If the result is smaller than the decimal length, zeroes are padded as required.
- If the decimal lengths are not equal:
  - The operand with the smaller decimal length is padded with zeros to make both lengths equal.
  - Subtraction is then performed as before, with proper formatting of the result, including zero padding where necessary.
- It removes unnecessary leading zeros.
- It ensures no trailing zeros remain using the `remove_extra_zeroes()` method.
- It returns the final result as a new `AFloat` object.

`mul(AFloat other)`

- Convert both operands to strings and ensure they contain a decimal point.
- If either operand doesn't have a decimal point, it adds one.
- It calculates the decimal lengths of both operands using the `decimal_digits()` method
- The decimal point is removed from both operands by converting them into integer-like strings.
- Leading zeros are also removed from both operands before multiplication.
- The operands are converted into `AInteger` objects and multiplied using `AInteger:: mul(AInteger odd)`.
- The result is stored and processed based on the total number of decimal places:
  - The total decimal length is the sum of the decimal lengths of both operands.
  - If the result has more digits than the total decimal length, the decimal point is inserted in the correct position.
  - If the result has fewer digits than the total decimal length, it pads the result with leading zeros and places the decimal point correctly.
- If the result is negative, the negative sign is handled properly and inserted in the correct place, with additional formatting for the decimal point.
- It removes unnecessary leading zeros.
- It ensures no trailing zeros remain using the `remove_extra_zeroes()` method.
- The result is truncated using the `truncate()` method to ensure the correct format.
- Finally, it wraps the result into a new `AFloat` object and returns it.

`div(AFloat other)`

- First, it ensures both operands have a decimal point.
- If either operand doesn't have a decimal point, `.0` is appended to it.
- The operands are parsed into `AFloat` objects to determine their decimal lengths using the `decimal_digits()` method.
- The decimal points are removed by converting both operands into integer-like strings.
- Leading zeros are removed from both operands before performing division.
- The division operation is performed by using `AInteger :: div (AInteger other)` for both the quotient and the remainder.
- If division by zero is attempted, an `IllegalArgumentException` is thrown.
- If the first operand is zero, the result is zero.
- The sign of the result is determined by the signs of the operands:
  - If one operand is negative, the result will be negative.
  - If both operands are negative, the result will be positive.
- The remainder is computed and processed to remove any negative sign and ensure it is handled correctly during division.
- The long division method is implemented to compute the result up to 30 decimal places:
  - The remainder is repeatedly divided by the divisor, and the quotient is appended to the decimal part of the result.
  - If the remainder becomes zero, the loop terminates.
- The result is formatted correctly:
  - The quotient and the decimal part are concatenated.
  - If there are extra zeros, they are added or removed as necessary, and the decimal point is inserted in the correct position.
- If the result is negative, the minus sign is handled correctly, and the number is formatted accordingly.
- Leading zeros are removed from the result using a custom method to ensure the correct formatting.
- After the result is calculated, it is verified by multiplying it back with the divisor to ensure no extra zeros are present. If necessary, extra zeros are removed.
- The final result is returned as an `AFloat` object.

MyInfArith.java

- It is a Java file that imports the package and runs test cases.
- The program begins by checking whether the first argument is `int` or `float` to determine the type of arithmetic operation.
- If the first argument is `int`
  - Both operands are parsed using `AInteger.parse()`.
  - Each character of the operands is validated to ensure it represents an integer.

  - The first character may be a minus sign (−) or a digit.
  - Subsequent characters must all be digits.
  - If any invalid character is found, an `IllegalArgumentException` is thrown.
  - Depending on the second argument, one of the following operations is performed using the `AInteger` class:
    * `add` for addition
    * `sub` for subtraction
    * `mul` for multiplication
    * `div` for division
  - The result is printed using `get()`.
  - If the second argument is invalid, an error message is printed describing the correct usage.

- If the first argument is `float`
  - Both operands are parsed using `AFloat.parse()`.
  - Each character of the operands is validated to ensure it represents a floating-point number.
  - The first character may be a minus sign (¡code¿'-'¡/code¿), a digit, or a decimal point.
  - All subsequent characters must be digits or a decimal point.
  - If any invalid character is found, an `IllegalArgumentException` is thrown.
  - Based on the second argument, the corresponding arithmetic operation is performed using `AFloat` methods:
    * `add` for addition
    * `sub` for subtraction
    * `mul` for multiplication
    * `div` for division
  - The result is displayed via the `get()` method.
  - If the second argument is not one of the supported operations, an error message with usage instructions is shown.

- If the first argument is neither `int` nor `float`, an error message is printed showing the correct usage syntax.

- How to use MyInfArith.java
  - First, compile `MyInfArith.java` along with `AInteger.java` and `AFloat.java`.
  - Use the command: `javac MyInfArith.java arbitraryarithmetic/*.java`
  - Then, run the program using: `java MyInfArith int/float add/sub/mul/div operand_1 operand_2`
  - Replace `int/float` with either `int` or `float`, depending on the operand type.
  - Replace `add/sub/mul/div` with the desired operation.
  - Replace `operand_1` and `operand_2` with your actual numeric values (e.g., `-12`, `3.14`).

my_exe

- It is a Python script used to run test cases.
- It uses `MyInfArith.java` to run the test cases.
- The script defines a class ¡code¿MyInfArith¡/code¿ that is initialized with command-line arguments.

- Inside `__init__`, it checks whether exactly four arguments are provided if not, it prints a usage pattern and exits.
- If the argument count is correct, it proceeds to `compile_and_run`.
- It uses the `os.system` command to compile all Java files in the `arbitraryarithmetic` directory along with `MyInfArith.java`.
- If the compilation fails (i.e., the return code is non-zero), it prints `Compilation failed` and stops execution.
- If compilation is successful then, It constructs the Java run command by joining the original four arguments with spaces.
- The Java program is executed using `java -cp .  MyInfArith ...` via `os.system`.
- You can run the file from the command `./my_exe int/float add/sub/mul/div operand1 operand2`.
- Replace `int/float` with either `int` or `float`, depending on the operand type.
- Replace `add/sub/mul/div` with the desired operation.
- Replace `operand_1` and `operand_2` with your actual numeric values (e.g., `-12`, `3.14`).

build.xml

- Defines a project named **"Arbitrary Arithmetic Operations"** with the default target `jar`.
- Sets both the source and build directories to the **current folder**, so `.class` files go alongside `.java` files.
- The `compile` target compiles `arbitraryarithmetic/*.java` into the same directory.
- The `jar` target creates `aarithmetic.jar` containing all `.class` files from the current directory tree.
- The `clean` target deletes the `.jar` file and all `.class` files from the directory.
- You can use the command `ant jar` to create the jar file.
- You can use the command `ant clean` to delete the jar file.

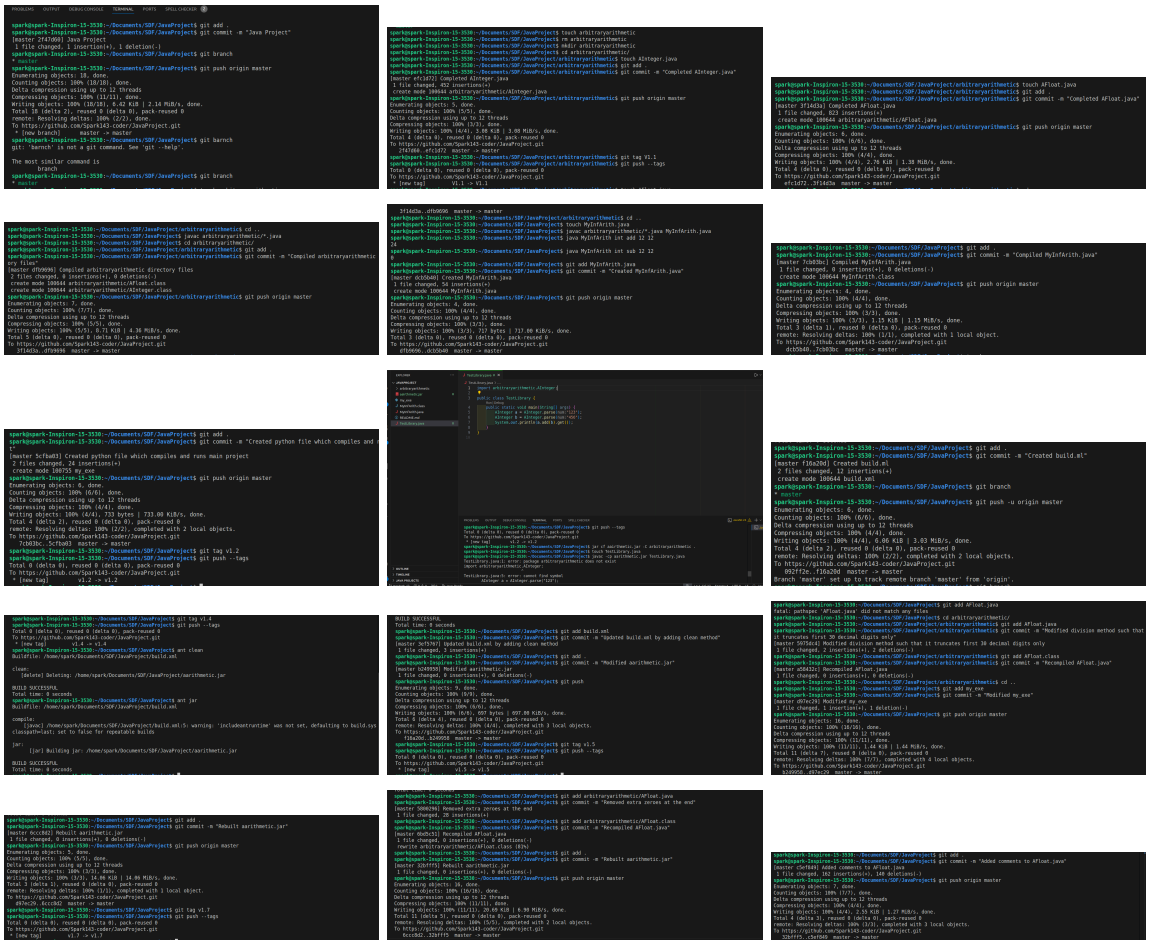
aarithmetic.jar

- The `aarithmetic.jar` library can be linked to any **executable/library**.
- Provides support for arbitrary-precision integer and floating-point arithmetic.
- Can be imported and used in any Java project via the classpath or build tool.
- Includes exception handling for common arithmetic errors.
- Tested via `MyInfArith.java` with various edge cases and corner scenarios to ensure correctness.
- To use this JAR File in your Java Program:
  - Ensure that `aarithmetic.jar` is in the same directory as your Java file or provide the correct path to it.
  - Use `import arbitraryarithmetic.AInteger;` and `import arbitraryarithmetic.AFloat;` at the beginning of your Java file to access the library's classes.

- To compile your Java file:
  - Run the following command:
  - `javac -cp aarithmetic.jar MyInfArith.java`.
  - This tells the compiler to include `aarithmetic.jar` in the classpath while compiling your program.
- To run the compiled Java program:
  - Use the following command:
  - `java -cp .:aarithmetic.jar MyInfArith int/float add/sub/mul/div operand1 operand2`
  - On Windows, replace the colon `:` with a semicolon `;` in the classpath:

Git Commits

Git Commits



limitations

- No support for special float values like `NaN,Infinity`.
- Only supports add, sub, mul, and div. No modulo, power, square root, or advanced math.
- The program prints errors directly rather than returning structured error types or exit codes.

- Big integer/float operations are likely string-based and not optimized.
- Performance degrades with input length. Operations on very large numbers (thousands of digits) will be slow.

Key Learnings

- Learned how to design and implement reusable classes like `AInteger`, `AFloat`, and `MyInfArith`.
- Understood the role of static methods (`like parse()`) for object creation.
- Learned how to store and manipulate integers and floating-point numbers as strings.
- Understood how to throw and catch exceptions `(IllegalArgumentException)` when inputs are invalid.
- Learned how to compile multiple Java files and manage packages.
- Understood how to compile code into a `.jar` and reuse it as a library using `-cp`.
- Developed a Python script to automate Java compilation and execution, learning inter-language automation.
- Improved debugging skills while testing operations on large or edge-case inputs.
- Practiced documenting usage patterns, limitations, and class functionality (potentially in HTML or LaTeX).